



Dynamic Programming Review

Introduction to Dynamic Programming

- **Definition:** Dynamic programming is an algorithmic technique that solves complex problems by breaking them down into simpler subproblems and storing the results to avoid redundant calculations.
- **Overlapping Subproblems:** The same smaller problems appear multiple times when solving the larger problem. Instead of recalculating these subproblems repeatedly, we store their solutions.
- **Optimal Substructure:** The optimal solution to the problem contains optimal solutions to its subproblems. This means we can build up the best solution by combining the best solutions to smaller parts.

Dynamic Programming Solutions

- **Memoization (Top-Down Approach):** Memoization stores the results of expensive function calls and returns the cached result when the same inputs occur again.

```
function climbStairsMemo(n, memo = {}) {  
  // If already calculated, return cached value  
  if (memo[n] !== undefined) {  
    return memo[n];  
  }  
  
  // Base cases: 1 way for 1 step, 2 ways for 2 steps  
  if (n <= 2) {  
    return n;  
  }  
  
  // Calculate once and store in memo for future use  
  memo[n] =  
    climbStairsMemo(n - 1, memo) +  
    climbStairsMemo(n - 2, memo);  
  
  return memo[n];  
}
```

- **Tabulation (Bottom-Up Approach):** Tabulation builds the solution from the ground up, filling a table with solutions to subproblems.

```
function climbStairsTabulation(n) {  
  if (n <= 2) {  
    return n;  
  }  
  
  // Create an array to store solutions from 0 to n  
  const dp = new Array(n + 1).fill(0);  
  
  // Base cases  
  dp[1] = 1; // 1 way to reach step 1  
  dp[2] = 2; // 2 ways to reach step 2
```



```
}  
  
    return dp[n];  
}
```

Real-World Applications Using Dynamic Programming

- **Route Optimization:** GPS systems use dynamic programming algorithms to find shortest paths between locations.
- **Text Processing:** Spell checkers and autocomplete features often rely on dynamic programming to calculate edit distances between words.
- **Financial Modeling:** Investment strategies and portfolio optimization frequently employ dynamic programming techniques.
- **Resource Allocation:** The knapsack problem and its variants appear in scheduling, budgeting, and resource management.

When to Use Dynamic Programming

You should consider using dynamic programming in the following scenarios:

- The problem can be broken down into overlapping subproblems.
- The problem exhibits optimal substructure.
- A naive recursive solution would involve repeated calculations.
- You need to optimize for time complexity at the cost of space complexity.

Assignment

- ☒ Review Dynamic Programming topics and concepts.

Submit

Ask for Help